

Educational Electricity Load Forecasting System with GUI

Final Technical Report

Authors:

David Santiago Téllez Melo
(20242020107)

Ana Karina Roa Mora
(20232020118)

Daniela Bustamante Guerra
(20241020131)

Andrés Felipe Correa Méndez
(20221020141)

Course: Systems Analysis and Design

Professor: Carlos Andrés Sierra, M.Sc.

Academic Program: Systems Engineering

Faculty of Engineering

Universidad Distrital Francisco José de Caldas

Bogotá, Colombia

Emails: {dstellezm, akroam, dabustamanteg, afcorrean}@correo.udistrital.edu.co

December 2025

Abstract

Abstract

This report presents an educational electricity load forecasting system developed as a Python desktop application. The system implements a complete forecasting workflow using a Random Forest machine learning model with a graphical user interface (GUI) built with Tkinter. Inspired by the Global Energy Forecasting Competition 2012 (GEFCom2012), the application allows users to load historical consumption data, train prediction models, visualize results, and generate future forecasts. The system follows Model-View-Controller (MVC) architecture for modularity and maintainability. It serves as both a functional forecasting tool and an educational platform for learning time series analysis and machine learning concepts.

Keywords: Electricity Load Forecasting, Random Forest, Python GUI, Tkinter, Educational Software, MVC Architecture

Contents

1	Introduction	3
1.1	Project Overview and Context	3
1.2	Integration of Previous Workshops	3
1.3	Project Objective	3
2	Methods	5
2.1	System Architecture and Design	5
2.2	Data Model	5
2.3	Preprocessing Model	5
2.4	Machine Learning Model Integration	6
2.5	Graphical User Interface	7
2.6	Controller Workflow	8
2.7	Future Demand Prediction	8
3	Results	9
3.1	System Testing	9
3.2	Kaggle Competition and Submission	9
3.3	Performance Evaluation	10
3.4	Simulation and Validation	11
3.5	Future Forecasting	12
3.6	Data Export	13
3.7	Project Management and Improvement	13
4	Experiments and Discussion	15
4.1	Experiments	15
4.2	Strengths	15
4.3	Limitations	15
4.4	Educational Impact	15
5	Conclusions and Future Work	17
5.1	Project Summary	17
5.2	Key Achievements	17
5.3	Educational Value	17
5.4	Future Work	17

List of Figures

2.1 MVC architecture applied to the forecasting system. 5

2.2 Overall pipeline of the implemented forecasting system, including data loading,
preprocessing, feature generation, model training, and prediction visualization. . . 7

2.3 Main graphical user interface of the electric load forecasting system. 8

3.1 Example of generated CSV submission compatible with Kaggle competitions. . . 10

3.2 Comparison between real and predicted electrical demand. 11

3.3 System behavior under different dataset sizes and conditions. 12

3.4 7-day future electricity demand forecast 13

3.5 Kanban board used for project management and task tracking. 14

Chapter 1

Introduction

1.1 Project Overview and Context

Electric load forecasting is a critical task for modern power systems, as it directly affects operational efficiency, economic performance, and service reliability. Electric utilities must continuously estimate future electricity demand in order to balance generation and consumption.

Producing less electricity than required may result in power outages, dissatisfied customers, and economic losses, while producing more electricity than necessary leads to wasted resources, unnecessary costs, and environmental impact. Therefore, accurate demand forecasting is fundamental for the efficient operation of electrical networks.

This course project is inspired by the Global Energy Forecasting Competition 2012 (GEF-Com2012), an international competition organized by the IEEE that focuses on short-term electricity load forecasting using historical data and contextual variables such as temperature. The objective of this project is to design and implement an end-to-end electric load forecasting system based on Machine Learning.

The system integrates data preprocessing, model training, prediction, visualization, and result export through a modular software architecture and a graphical user interface.

1.2 Integration of Previous Workshops

This project consolidates the main results of all workshops developed during the course:

- **Workshop #1 – Competition Analysis:** Analysis of a Kaggle-style energy forecasting competition, including dataset structure, objectives, evaluation criteria, and submission format.
- **Workshop #2 – System Design and Architecture:** Definition of the system architecture using the Model–View–Controller (MVC) pattern.
- **Workshop #3 – Management and Improvement:** Application of project management strategies, task organization, and iterative system refinement.
- **Workshop #4 – Simulation and Validation:** Evaluation of system behavior under varied datasets, edge cases, and performance scenarios.

1.3 Project Objective

In this context, the objective of this work is to develop an educational and functional system for electric load forecasting using Machine Learning techniques. Specifically, a Random Forest

regression model is employed to learn consumption patterns from historical data and generate demand predictions. The system is designed with a graphical user interface that allows users to interactively load data, train models, visualize results, and generate future demand forecasts.

Chapter 2

Methods

2.1 System Architecture and Design

The system is implemented using the Model–View–Controller (MVC) architectural pattern. This pattern separates the application into three components:

- **Model:** Handles data loading, preprocessing, feature engineering, and machine learning algorithms.
- **View:** Provides the graphical user interface (GUI), including buttons, plots, logs, and visual feedback.
- **Controller:** Coordinates the workflow and connects user actions with model operations.

This separation improves maintainability, scalability, and clarity, allowing each component to evolve independently.

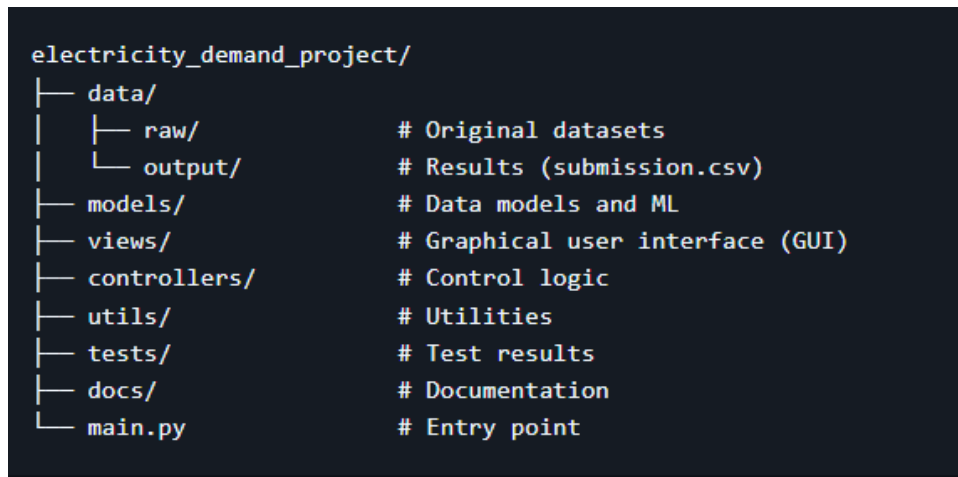


Figure 2.1: MVC architecture applied to the forecasting system.

2.2 Data Model

The Data Model is responsible for loading datasets stored in CSV format and validating their structure. Each dataset must contain the following mandatory columns: **datetime**, **load**, and **temperature**. The model verifies that the file exists, is not empty, and has the required columns before returning the data for further processing.

2.3 Preprocessing Model

The preprocessing stage prepares raw data for machine learning and consists of three main tasks:

- **Missing value imputation:** Missing values in load or temperature are replaced using the mean of the corresponding column.
- **Temporal feature generation:** From the datetime column, additional features are extracted, including hour of day, day of week, day of year, and a binary weekend indicator.
- **Temperature normalization:** Temperature values are scaled to the range $[0, 1]$ using Min-Max normalization.

These transformations allow the model to capture temporal consumption patterns and operate with variables on comparable numerical scales.

2.4 Machine Learning Model Integration

A Random Forest Regressor was selected as the core forecasting model due to its robustness, ability to capture nonlinear relationships, and resistance to overfitting. The model consists of 100 decision trees, and the final prediction is computed as the average of all individual tree predictions.

The input features used for training include hour of day, day of week, day of year, weekend indicator, and normalized temperature. These features allow the model to capture temporal and environmental patterns in electricity consumption.

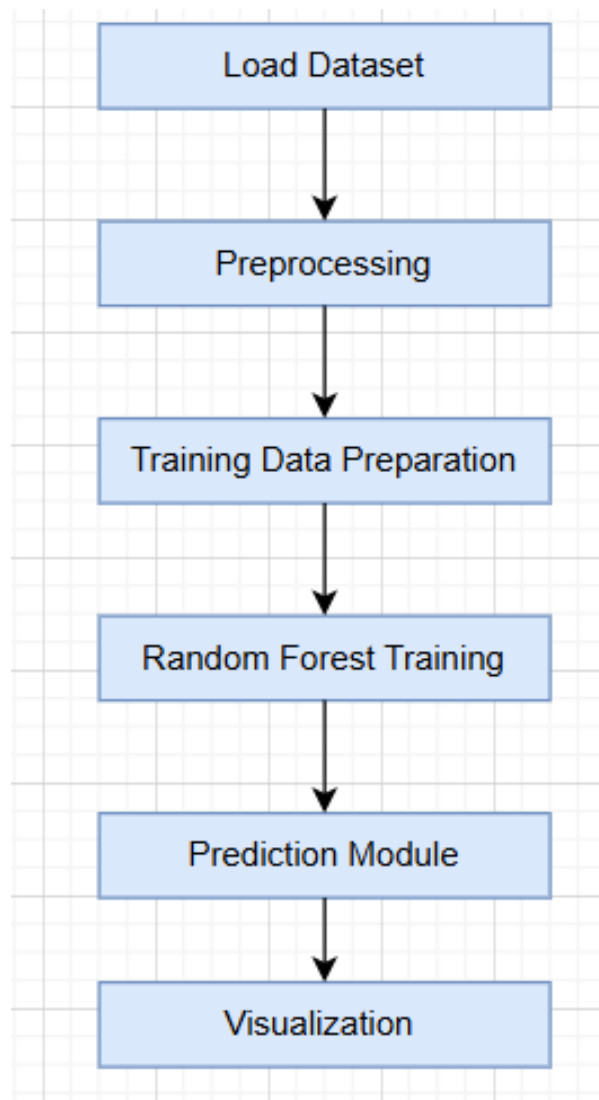


Figure 2.2: Overall pipeline of the implemented forecasting system, including data loading, preprocessing, feature generation, model training, and prediction visualization.

2.5 Graphical User Interface

The View layer is implemented using the Tkinter library. The graphical interface provides buttons for loading datasets, training the model, exporting results, and generating future predictions. It also includes a text area for system messages and an embedded Matplotlib area to visualize real versus predicted demand through line plots.

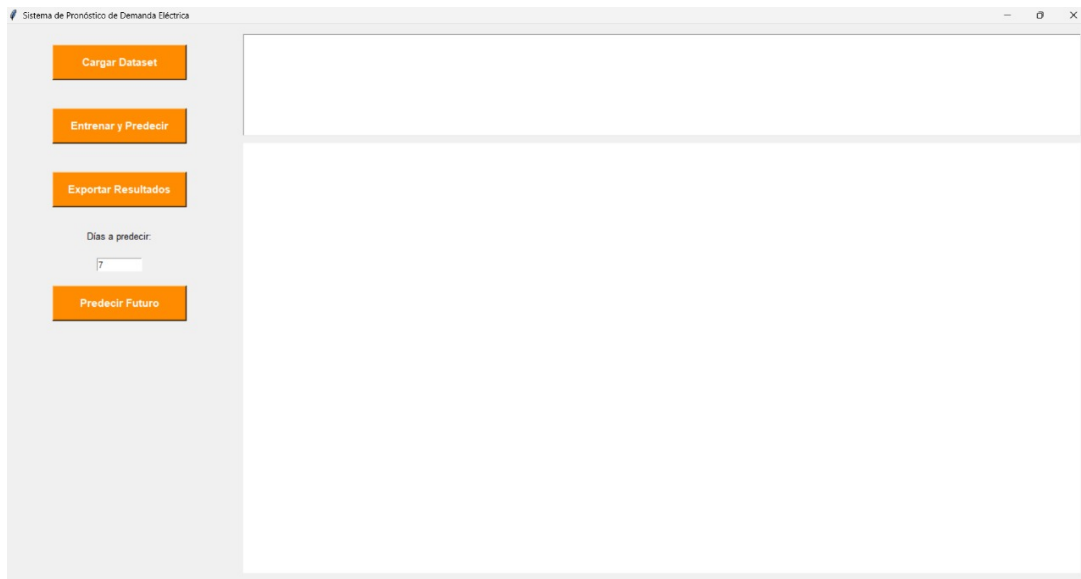


Figure 2.3: Main graphical user interface of the electric load forecasting system.

2.6 Controller Workflow

The Controller manages the interaction between the user interface and the underlying models. When the user loads a dataset, the controller triggers data validation and preprocessing. During model training, it extracts the input features and target variable, displays a progress indicator, and generates predictions. For future forecasting, it coordinates the generation of future timestamps, preprocessing, prediction, visualization, and automatic export of results.

2.7 Future Demand Prediction

To generate future demand forecasts, the system creates hourly timestamps for a user-defined number of days. Since future temperature values are unknown, synthetic temperatures are generated using a normal distribution based on the historical mean and standard deviation. These values are processed using the same preprocessing pipeline and passed to the trained model to obtain future demand predictions.

Chapter 3

Results

3.1 System Testing

The system was tested using three datasets:

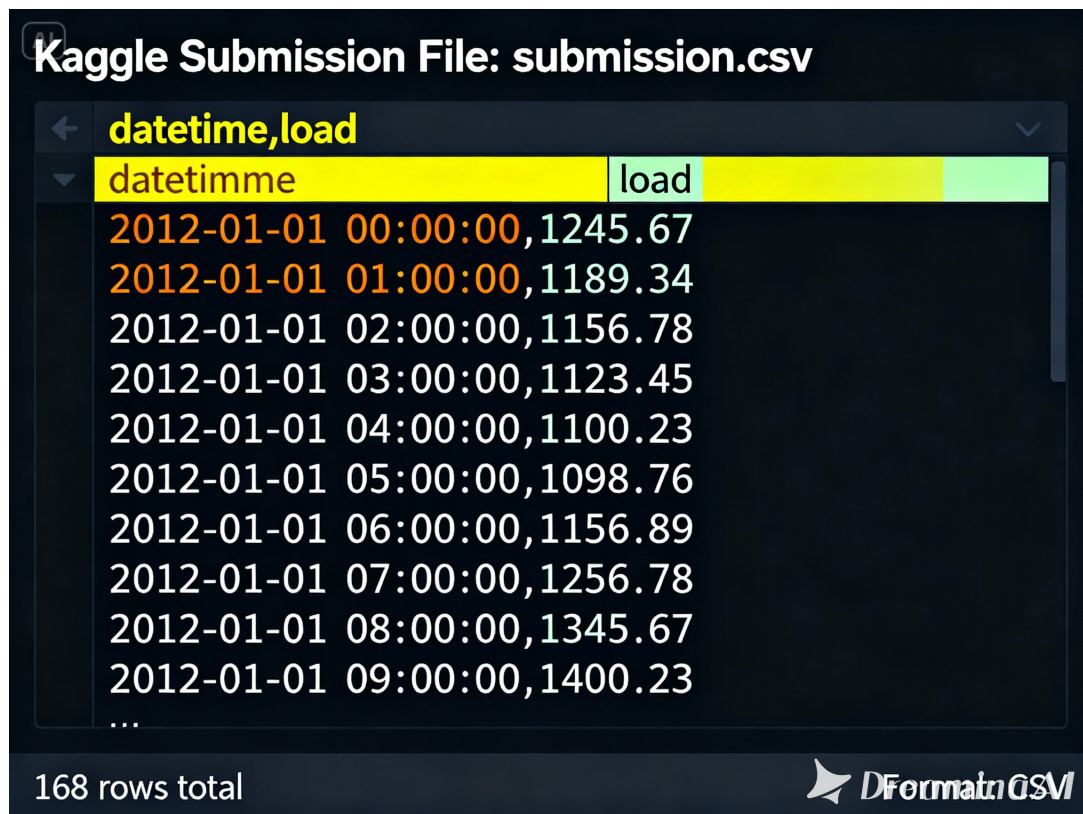
Dataset Type	Purpose
Small dataset	Quick validation and debugging
Large dataset	Performance evaluation
Extreme dataset	Testing preprocessing robustness

Table 3.1: Datasets used for system testing

3.2 Kaggle Competition and Submission

Based on the analysis conducted in Workshop #1, a valid Kaggle-style submission was generated. The system automatically produces a CSV file containing timestamps and corresponding load forecasts, following Kaggle submission format requirements.

The submission workflow demonstrates how the developed system can be used to automatically generate or refine competition-ready predictions.



Kaggle Submission File: submission.csv

datetime	load
2012-01-01 00:00:00	1245.67
2012-01-01 01:00:00	1189.34
2012-01-01 02:00:00	1156.78
2012-01-01 03:00:00	1123.45
2012-01-01 04:00:00	1100.23
2012-01-01 05:00:00	1098.76
2012-01-01 06:00:00	1156.89
2012-01-01 07:00:00	1256.78
2012-01-01 08:00:00	1345.67
2012-01-01 09:00:00	1400.23
...	

168 rows total

DFormat CSV

Figure 3.1: Example of generated CSV submission compatible with Kaggle competitions.

3.3 Performance Evaluation

Model performance was evaluated primarily through visual comparison of real versus predicted demand curves. As shown in Figure 3.2, the Random Forest model captures daily and weekly consumption patterns with reasonable accuracy.

Compared to baseline approaches explored in earlier workshops, the model produces smoother and more stable predictions, particularly during peak demand periods.

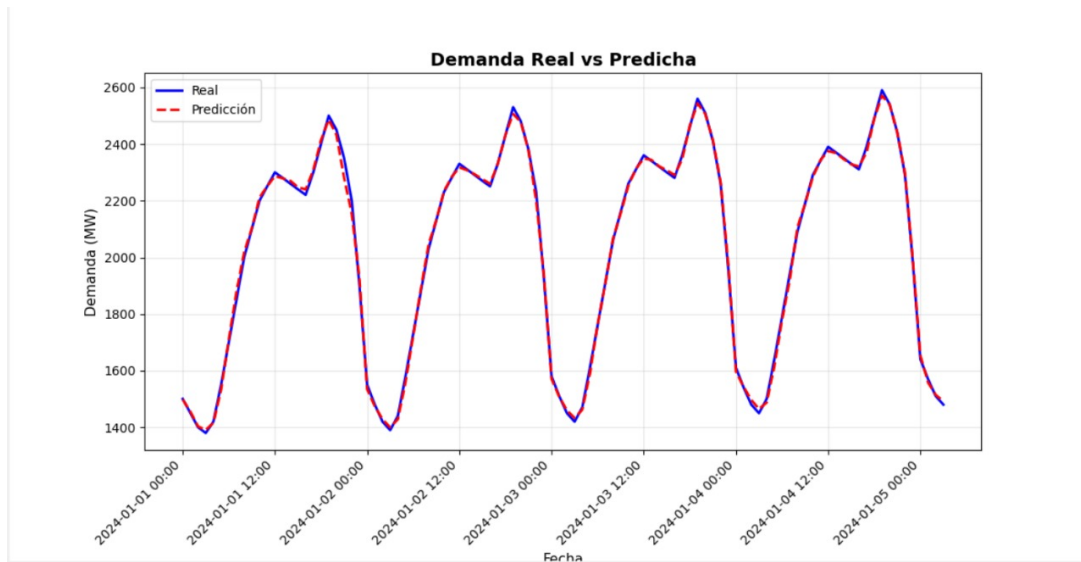


Figure 3.2: Comparison between real and predicted electrical demand.

3.4 Simulation and Validation

Simulation strategies from Workshop #4 were applied to assess robustness and scalability. The system was tested using datasets of different sizes and quality levels.

The preprocessing pipeline successfully handled missing values and extreme cases, while execution times remained acceptable even for larger datasets. The system maintained stable behavior under unexpected user interactions and incomplete data inputs.



Figure 3.3: System behavior under different dataset sizes and conditions.

3.5 Future Forecasting

A seven-day forecasting scenario was executed, producing 168 hourly demand predictions. Figure 3.4 shows the generated future predictions.

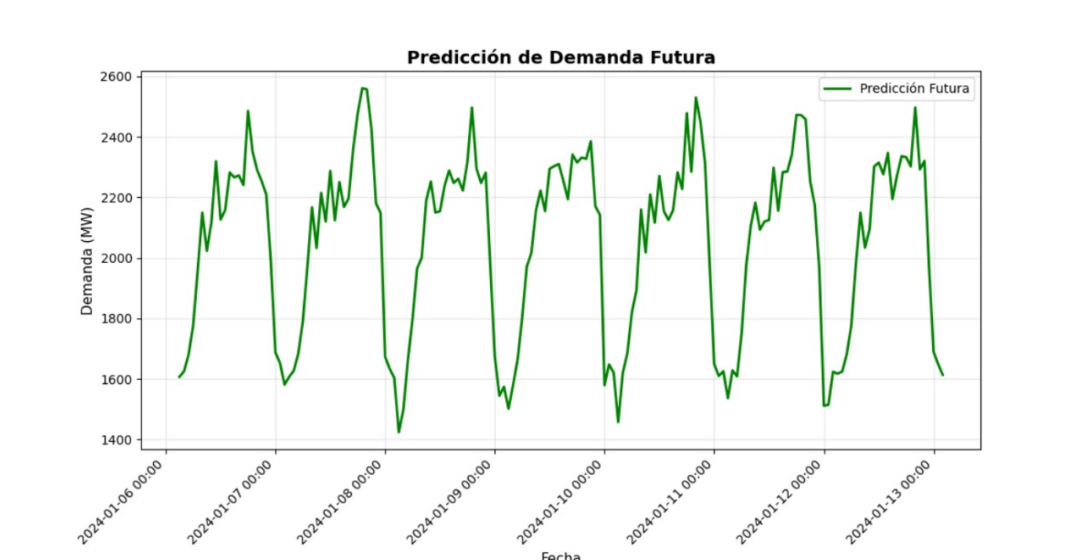


Figure 3.4: 7-day future electricity demand forecast

3.6 Data Export

All results were successfully exported in CSV format:

- `submission.csv`: Contains historical predictions
- `future_predictions.csv`: Contains future forecasts

3.7 Project Management and Improvement

Project management strategies defined in Workshop #3 were applied throughout development. A Kanban board was used to organize tasks related to data handling, model integration, interface development, testing, and documentation.

Iterative improvements were introduced based on testing results and feedback, leading to enhanced preprocessing robustness, clearer visualization, and improved user experience.



Figure 3.5: Kanban board used for project management and task tracking.

Chapter 4

Experiments and Discussion

4.1 Experiments

Several experiments were conducted to explore potential improvements to model performance. These included testing different feature combinations, modifying the number of trees in the Random Forest model, and experimenting with synthetic temperature generation for future forecasting.

Although advanced hyperparameter optimization was not performed, these experiments provided insight into the sensitivity of the model to feature selection and data variation.

4.2 Strengths

The developed system demonstrates strong educational and practical value:

- **Modular MVC architecture** improves clarity and extensibility
- **Graphical interface** allows users without advanced programming skills to interact with the forecasting pipeline
- **Complete workflow** integration of data loading, preprocessing, modeling, visualization, and export
- **Educational transparency** makes machine learning concepts accessible
- **Integration of course workshops** demonstrates comprehensive learning application

4.3 Limitations

However, the system has limitations:

- Relies on a **single machine learning algorithm** (Random Forest only)
- Does not compute **quantitative evaluation metrics** such as RMSE or MAE
- Only **temperature** is considered as an external explanatory variable
- Future predictions use **synthetic temperature data**

4.4 Educational Impact

Despite these limitations, the system fulfills its objective as a learning-oriented forecasting tool. It provides:

- Hands-on experience with forecasting concepts
- Understanding of complete ML workflows
- Practical software engineering examples
- Real-world application context
- Integration of theoretical concepts with practical implementation

Chapter 5

Conclusions and Future Work

5.1 Project Summary

This project presents a functional and educational system for electric load forecasting based on Machine Learning. By combining a Random Forest model with an MVC architecture and a graphical interface, the system enables intuitive interaction with real forecasting processes. The project successfully integrates software engineering principles with machine learning techniques, demonstrating a complete forecasting workflow.

5.2 Key Achievements

- Successful implementation of complete forecasting pipeline
- User-friendly GUI for educational access
- Robust data processing with validation
- Effective visualization of results
- Future forecasting capability
- Integration of all course workshop deliverables
- Generation of Kaggle-compatible submission files

5.3 Educational Value

The application successfully supports historical analysis, real-time visualization, and future demand prediction. Its design makes it suitable for academic environments and introductory studies in energy forecasting. The project demonstrates how theoretical concepts from Systems Analysis and Design can be applied to real-world data science problems.

5.4 Future Work

Future improvements could include:

- Incorporating additional machine learning models (XGBoost, LSTM, etc.)
- Adding quantitative evaluation metrics (RMSE, MAE, MAPE)
- Including more explanatory variables (humidity, holiday indicators, etc.)
- Implementing model comparison features

- Enhancing visualization options
- Extending the system to web-based or cloud environments
- Adding hyperparameter optimization capabilities

Bibliography

- [1] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Available: <https://scikit-learn.org/>
- [2] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. Available: <https://link.springer.com/article/10.1023/A:1010933404324>
- [3] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010. Available: <https://pandas.pydata.org/>
- [4] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. Available: <https://matplotlib.org/>
- [5] Python Software Foundation, “Tkinter — Python interface to Tcl/Tk,” Python documentation. Available: <https://docs.python.org/3/library/tkinter.html>
- [6] Kaggle, “Search: Global Energy Forecasting Competition 2012,” Kaggle Datasets Search. Available: <https://www.kaggle.com/search?q=Global+Energy+Forecasting+Competition+2012>